

Mechanical Advantage: 2026 Robot Code

- ☐ Ensure you've got NI's **FRC Game Tools** and **WPILib** installed. **LabVIEW Base Installer** is not necessary.
- ☐ Clone or download the code at [Mechanical-Advantage/RobotCode2026Public](#).
- ☐ Log into Github... This isn't necessary here, but please ensure that you have that set up.
 - The **Darwin Binder** explains controls for their project and covers how they design & structure autos.

Autonomous, Controls and What To Expect To See

What each **Auto Routine** choice means isn't critical to understand. Once the AdvantageScope layout is set, you'll see trajectories in the **Vis3D** and **Vis2D** tabs.

- Hit **Left Trigger** to align with the target. The docs in their robot binder imply that this button also launches. **Right Trigger** will force launch.
- Use **A** to align the robot to the towers.
- Controls shown in the **Binder** that don't point to an X-Box button are for the operator AFAIK.

Similar to our Robot's code, moving the robot changes the target, but they approached this differently. In the `2026-09-12-mech-adv.j` layout, which is imported later, the current target shows up as the **DuckBot**.

There are some other tabs set up in this layout:

Tab	Subsystem(s)	Purpose
Vis3D	Drive, Launcher	Show robot position & targeting. (Later, also render vision system)
Vis2D	Drive, Launcher	Simpler view on robot position
Flywheel	Hood, Flywheel, Hopper	Check subsystem states, setpoints and measurements
Intake	Slam, Slamttake, Kicker	Check subsystem states, setpoints and measurements
Energy		Track power usage over time
Trace		How long does each subsystem's periodic() require?
Swerve	Drive	Visualize vectors for swerve modules
X		Visualize X-Box Controller State (useful during replays)
CAN	Almost Surely Every Subsystem	This is the most important tab here!

Run the Robot Simulator

Before running anything, copy the `*.json` files from `./sim` into the main project directory. WPILib looks for these when you start a project and you'll need to delete/replace them otherwise. These define the look & feel of the **Robot Simulator** application, where you can quickly set the auto routines instead of using **Elastic Dashboard**

- ☐ Copy the `*.json` files
- ☐ Make sure their project is building
- ☐ Compile with WPILib: **Build Project** and wait for it to complete
- ☐ Run with WPILib: **Simulate Project** and check the **Sim GUI** box when it's ready

Mech. Adv. uses **choreo** for autonomous, instead of **PathPlanner**.

- ☐ Drag an X-Box controller over to `joystick[0]`. Ignore the "second controller" warning (operator controls).
- ☐ Look through their **Auto Routine** choices in the **Robot Simulator**.
- ☐ Toggle the **Robot State** from **Disconnected** to **Autonomous**. This won't seem to do much until AdvantageScope is open.

Note: You'll need an X-Box controller. Otherwise, this will require implementing an alternative in Java. Whether that's simple depends on the project, but most teams settle on either PS or X-Box.

Don't Install Python

The directions mention installing **python3**, but... installing **python** up on Windows is similar to setting up **git**: there are many ways to do it. If you try several methods to install it, but don't clean them up, this can cause problems later.

AFAIK, the **python** code is only required for:

hub_counter A small Raspberry Pi project deployed inside the hub that counts shots. It may support simulation, but seems to be complicated.

northstar Their custom vision that runs on Mac Mini's. This is a C/C++ project with a python wrapper. Sim may be possible, but difficult.

Setup Elastic

After getting the robot code building and running, set up Elastic Dashboard. It's quick.

- ☐ Find `2026_Elastic.json` at the root of their source code and import it.
 - Use **Ctrl + Shift + K** to connect to your simulator.

During practice and competition, we will set the **Auto Routine** in **Elastic Dashboard**. When simulating, this could be set in **Robot Simulator**, but only if the `simgui.json` file is set up to provide dropdowns for those `/SmartDashboard/Auto` values. This does mean you need one less application to run the sim.

Setup AdvantageScope

Import a layout

- ☐ Download 2026-09-12-mech-adv.json

Import their model

- ☐ Move assets to \$HOME/.config/AdvantageScope/userAssets.

Their instructions say to change the `Custom Assets` location. Don't do that. It may prevent game pieces from showing up IIRC and the setting is sticky. i.e. You'll need to reset it later.

Instead, the fastest way to find it: `App > Show Assets Folder`.

- Use `~/.config/AdvantageScope/userAssets` for Linux.
- Use `%APPDATA%\AdvantageScope\userAssets` for Windows.

This is complicated and a few things can go wrong. Some examples:

1. Can't have two of the same bot. In `./Robot_Darwin/config.json`, `.name` must be unique.
2. Folders must have specific name (e.g. `Robot_` must prefix a robot).

Connect to the Simulator

- ☐ Make sure the Simulator Source is `NetworkTables 4`
 - Go to `File > Connect to Simulator > NetworkTables 4`
- Use `Ctrl + Shift + K` to connect to your simulator

Navigating the Code

When you want to know how something works, find a key string and search VS Code for it. This isn't always the most direct way, but sometimes you find things you weren't looking for. A good bet is to:

- Look for the name of a logged value in AdvantageScope, like `LaunchCalculator` or `FuelSim`, which I needed to find.
- Then use `Ctrl + Shift + F` in VSCode. Try to skip results that start with `import` unless there are many lines in that file with the result.
- Results that show up in `RobotContainer` are pretty important: this is the main entrypoint for your robot's program. This shows the differences between the simulated robot and the actual robot.
- Usually you'll want to find the main subsystem where things are defined.

A **subsystem** is a common pattern in low-level programming. The Linux kernel also has subsystems. A subsystem is intended to provide a thin interface to otherwise complicated components. In the terminology of design patterns, the thin interface is called a "facade." This ensures the sub-system is responsible for handling its own logic & components – so things from the outside can't improperly interfere with them.

- For Linux, this helps with the security & integrity of running programs.
- For robotics, the subsystem interfaces help with timing and tracing copies of values.

This is in addition to organizing code, which is obvious when first exploring the idea of subsystems. Later, it's helpful that code outside of the subsystems will use a common set of timings where you're more certain about when measurements were collected.

BoVLB's FRC Tips

These are useful for explaining Command-based programming in WPILib, especially as the commands relate to subsystems and triggers.

- **Best Practices for Command-based Programming** covers how to isolate commands to subsystems, so they can be composed into commands groups.
- **Commands**: this describes subsystem requirements, which limit how subsystems are associated to commands.
- **CommandScheduler**: a deep dive into the command scheduler, which is advanced but may be useful later.

Subsystem requirements are important to understand up front when you're designing how your controller's triggers. These are usually declared in `RobotContainer`. Each subsystem can only be "required to" one concurrently running command.

Instead of using the **Command Factory** pattern in these articles, our 2026 code changed behavior based on the subsystem's current state – i.e. we used a **State Machine** which isn't a mutually exclusive design, per se. There are many ways to go about designing your robot – many "programming paradigms", like functional, imperative, etc. None of them are 100% best suited for all situations, but mixing them in the wrong way leads to problems later. This is important to remember when referring to those articles (our existing codebase won't mix well with the Command Factory approach). Both approaches are valid and most FRC seasons will need a state machine somewhere.

The Java language helps guide your design. Sometimes compiler warnings can be frustrating, but the right design will help you avoid the difficulties mentioned above. Python relaxes these constraints, but code that mixes these styles can still become messy. Only experience can help avoid this. Focusing too much on it without experience isn't necessarily useful. Once a project includes about 25-75 classes or interfaces, the design gains some inertia.